

Code Assessment of the M Portal Lite Smart Contracts

September 12, 2025

Produced for



by



Contents

1	Executive Summary	3
2	Assessment Overview	5
3	Limitations and use of report	8
4	Terminology	9
5	Open Findings	10
6	Informational	13
7	Notes	14

1 Executive Summary

Dear all,

Thank you for trusting us to help M0 with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of M Portal Lite according to [Scope](#) to support you in forming an opinion on their security risks.

M0 implements M Portal Lite, a one-to-many bridge that primarily allows transfers of the tokens from the M ecosystem between the Hub (Mainnet) and some Spoke chains (L2), via the Hyperlane protocol.

The most critical subjects covered in our audit are asset solvency, functional correctness, and precision of arithmetic operations. Considering the restricted scope defined in [Assessment Overview](#), we find that that the security regarding all the aforementioned subjects is high.

The general subjects covered are documentation, gas efficiency, and the integration of the wrapper into the existing system. Security regarding all the aforementioned subjects is generally good.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	3
Risk Accepted	3

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the M Portal Lite repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	13 Aug 2025	ae5baef7d5cbbc326b65b8be92a38d2eebb5c3b4	Initial Version

For the solidity smart contracts, the compiler version 0.8.26 was chosen.

The scope of the review is limited to the changes related to the SwapFacility integration (1b3ce427a82a9680998423bb17bcea21ede11f11) and the reentrancy lock mechanism (ae5baef7d5cbbc326b65b8be92a38d2eebb5c3b4) in the diff from 099ea8ee2b75c789866dd095a6362273e44263a4 in the following files:

```
src:
  HubPortal.sol
  Portal.sol
  SpokePortal.sol
```

For chains that do not support the TLOAD and TSTORE opcodes, the files implementing the reentrancy lock mechanism in scope of the review are the following in <https://github.com/m0-foundation/m-portal-lite/pull/34>:

```
src:
  Portal.sol
libs:
  Locker.sol
  ReentrancyLock.sol
```

2.1.1 Excluded from scope

Any contracts not explicitly listed above are out of the scope of this review. Third-party libraries are out of the scope of this review. The Hyperlane bridge protocol is assumed to work correctly. Any change outside of the scope defined above is out of the scope of this review. In particular, changes in the access control, contracts upgradeability and bridging logic are out of scope.

In this report, we assume the codebase at commit 099ea8ee2b75c789866dd095a6362273e44263a4 to be secure, and the review is focused on the changes applied to it. Therefore, any vulnerability present at commit 099ea8ee2b75c789866dd095a6362273e44263a4 might still be present in the codebase. |

2.2 System Overview

This system overview describes the initially received version (**Version 1**) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

M0 offers M Portal Lite, a one-to-many bridge that primarily allows transfers of the tokens from the M ecosystem between the Hub (Mainnet) and some Spoke chains (L2), via the Hyperlane protocol. M Portal Lite will see a HyperlaneBridge (out-of-scope) deployed on the Hub and the Spoke chains will manage the messages going in and out of the Hyperlane mailbox. The bridge is then connected to a Portal, either HubPortal or SpokePortal, responsible for locking/burning MToken on the source chain and unlocking/minting MToken on the destination chain. Not only tokens can be bridged, some data such as the earner index and other updates to a registrar key or list can be bridged from the Hub to the Spokes.

2.2.1 Portal

This is an abstract contract responsible for handling the logic common to all the portals, the specialized logic will be implemented by HubPortal and SpokePortal. It is also the entrypoint for users to initiate a transfer of some tokens from the M ecosystem and for the bridge to handle a message coming from Hyperlane. When bridging a token from the M ecosystem that is not the MToken, it will first be swapped out in the SwapFacility (see our [M Extensions Report](#)). The contract is deployed behind an UUPS proxy and can be upgraded. The contract can also be paused, which will block users from bridging out, but messages coming in from the bridge will still be processed.

Users can call the following functions:

- `transfer()`: bridge some amount of MToken to the destination chain through Hyperlane. The tokens are pulled from the caller and either locked or burned (`_burnOrLock()`), the exact implementation depends on the source chain. This function will revert if the contract is paused or if the destination chain is not supported.
- `transferMLikeToken()`: bridge some amount of a token from the M ecosystem to the destination chain through Hyperlane. The destination token can be the same or a different token (from M ecosystem) than the source token. The tokens are pulled from the caller, swapped to MToken via the SwapFacility and either locked or burned (`_burnOrLock()`), the exact implementation depends on the source chain. This function will revert if the contract is paused, if the destination chain is not supported, or if the source-to-destination path is not explicitly allowed.

The bridge entrypoint is:

- `receiveMessage()`: if the message is not for token bridging (Index, Key or List) the message will be handled by the implementation if `_receiveCustomPayload()` is overridden. If the message is for token bridging (Token), the required amount of MToken is either unlocked or minted (`_mintOrUnlock()`), the exact implementation depends on the destination chain, and swapped into the desired destination token via the SwapFacility if needed. If the call to the SwapFacility fails, the raw MToken is sent to the recipient.

2.2.2 HubPortal

The HubPortal implements Portal on Mainnet. It tracks the principal amount bridged to each of the Spoke chains. Its implementation of `_burnOrLock()` increases the principal amount locked for the destination chain. Its implementation of `_mintOrUnlock()` reduces the principal amount locked for the source chain and sends the required amount of MToken to the recipient if the destination token is MToken.

Users can also initiate the bridging of data with the following functions:

- `sendMTokenIndex()`
- `sendRegistrarKey()`
- `sendRegistrarListStatus()`

2.2.3 SpokePortal

The SpokePortal implements Portal on an L2. Its implementation of `_burnOrLock()` burns the desired amount of MToken on the Spoke chain. Its implementation of `_mintOrUnlock()` mints the desired amount of MToken on the Spoke chain. The contract also overrides `_receiveCustomPayload()` to mirror data coming from the Hub.

2.3 Trust Model

User: not trusted. Owners of the contracts: fully trusted. They are trusted to manage the proxies implementation in the best interest of the users and in a non-adversarial manner. They are also expected to pause and unpause the contract in the best interest of the users. If malicious or compromised, they can upgrade the implementations of the portals and steal all the locked MTokens. Pausers of the contracts: semi-trusted. They are expected to pause and unpause the contract in the best interest of the users. If malicious or compromised, can prevent users from bridging their tokens from and to the Hub chain until the owner revokes and replaces the pausers. Hyperlane: fully trusted. If malicious or compromised, can deliver arbitrary messages to either end of the bridge and mint freely mint MToken on the Spoke chain, or steal all the funds locked on the Hub. Spoke chains sequencers: fully trusted. If malicious or compromised, can censor users' transactions.

2.3.1 Other assumptions

- The HubPortal and SpokePortal will have the `M_SWAPPER_ROLE` or be allowed to call `swapInM()` and `swapOutM()` for the supported destination tokens.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe our findings. The findings are split into these different categories:

- **Design**: Architectural shortcomings and design inefficiencies
- **Correctness**: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	3

- Discrepancy Between Tracked and Actual Principal **Risk Accepted**
- Discrepancy of Index When Earning Is Disabled **Risk Accepted**
- Inconsistent Storage Locations **Risk Accepted**

5.1 Discrepancy Between Tracked and Actual Principal

Design **Low** **Version 1** **Risk Accepted**

CS-MPRTL-LT-005

In the HubPortal, the principal amount that is tracked for each Spoke might diverge from the actual transferred principal amount of MToken.

Specifically, the tracked principal for a given Spoke chain is decremented as follows:

```
uint256 totalBridgedPrincipal = bridgedPrincipal[sourceChainId_];
uint256 principalAmount = IndexingMath.getPrincipalAmountRoundedDown(uint240(amount_), _currentIndex());

// Prevents unlocking more than was bridged to the Spoke
if (principalAmount > totalBridgedPrincipal) revert InsufficientBridgedBalance();

unchecked {
    bridgedPrincipal[sourceChainId_] = totalBridgedPrincipal - principalAmount;
}
```

Where principalAmount, subtracted from bridgedPrincipal, is given by converting amount_ to principal. Note that the result of this operation is rounded down.

As a consequence, it is possible to bridge back some non-zero amount and not decrease the internally tracked principal (bridgedPrincipal). In fact, the bridge can theoretically be drained if an adversary can mint arbitrary unbacked MToken on a spoke chain ("Spoke is compromised"). This attack would be extremely expensive as the bridge will have to be drained by multiple transactions of index / EXP_SCALED_ONE wei each.

Risk Accepted: M0 has accepted the risk stating:

[...] we consider the described attack highly unlikely, as an attacker would spend more on gas and bridging fees than they could ever extract.

5.2 Discrepancy of Index When Earning Is Disabled

Correctness **Low** **Version 1** **Risk Accepted**

CS-MPRTL-LT-001

If the governance removes the `HubPortal` from the earners list, anyone can trigger a call to stop the portal from earning more yield in `MToken` in two ways:

1. Call `disableEarning()` in the `HubPortal` wrapper contracts.
2. Call `stopEarning()` in `MToken` contract.

The first option is the correct way to stop the wrapper earning more yield and ensure that the reported index is correct. However, since anyone can call `stopEarning()` in the `MToken` contract, the second option is also possible. In this case, the index bridged to the Spoke chains will be too big and will still grow until `disableEarning()` is executed. Theoretically, any delay might create solvency issues for the `HubPortal` as the yield accrued might get smaller than the yield accrued on the Spoke chains. Hence last users cannot bridge back their tokens and some of the tokens on the Spoke chains will be unbacked.

Risk accepted:

M0 accepted this risk and stated:

Removing ``HubPortal`` from the Earner list is a highly unlikely event. If it does occur, the M0 engineering team will monitor the Registrar list update and execute the ``disableEarning()`` call accordingly.

5.3 Inconsistent Storage Locations

Design **Low** **Version 1** **Risk Accepted**

CS-MPRTL-LT-002

The contracts `Portal`, `HubPortal`, and `SpokePortal` currently define their storage variables in a linear layout, without using the ERC-7201 storage slot standard. On the other hand, the parent contract `PausableOwnableUpgradeable`, uses ERC-7201.

This inconsistency in storage layout approaches may introduce potential upgradeability risks. Specifically, future upgrades may inadvertently overwrite storage variables or cause storage collisions.

Risk accepted:

M0 accepted this risk and stated:

If future upgrades require more changes to the storage, we'll consider adopting ERC-7201 storage slot standard everywhere and migrating storage values during the upgrade.

6 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

6.1 Non-earning HubPortal Should Be Paused

Informational **Version 1** **Acknowledged**

CS-MPRTL-LT-003

If earning is disabled for HubPortal and the contract is not paused, the rounding error due to transfers to the Hub will not be covered by the yield accrual and potentially render the HubPortal insolvent.

Acknowledged:

M0 has stated:

Removing HubPortal from the Earner list is a highly unlikely event. If it does occur, HubPortal could be either paused or upgraded to remove rounding error subsidy on transfers.

6.2 Not All Yield Will Be Claimed

Informational **Version 1** **Acknowledged**

CS-MPRTL-LT-004

As it is unlikely that the MToken holders on the Spoke chains will be earning, the HubPortal (assumed to be earning) will be accruing more yield compared to these holders. As a consequence, the MToken balance of the HubPortal will be greater than the amount bridged on the Spoke chain. The difference between those amounts will be locked in the HubPortal.

As a note, even if all the holders on Spoke chains are earners, because of time delay induced by the bridge, the index on the Hub chain will always be strictly bigger than indices on the Spoke chains as long as the HubPortal is earning.

Acknowledged:

M0 has stated:

We expect that most of the bridged M tokens will be in earning contracts, since all governance-approved M extensions are earners. We're aware that the excess yield locked in HubPortal isn't recoverable, and we're comfortable with this as we don't anticipate it being a significant amount.

7 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

7.1 HubPortal Removed From Earners

Note Version 1

If the `HubPortal` is removed from the earners list, the index it bridges will stop being updated. This effectively disables earning on all the connected Spoke chains.

7.2 Users Should Make Sure Index Is Not Stale

Note Version 1

Before bridging back from Spoke to Hub, earning users should make sure the current index on the Spoke chain is not stale compared to the `HubPortal` current index. If the difference is significant, they might give up on a non-negligible yield. The index on the Spoke chains can be updated with `HubPortal.sendMTokenIndex()`.